

Design, Implementation and Performance Analysis of an Autonomous Cherokee 4WD Robot

Course: Sensors and Network Devices

Team: Zhang Junqi & Houda Mesly

You can check the source code at: https://github.com/IndanthronBlue/Ubinet/tree/main/sensors_network_devices/robot

1. Introduction:

1.1 Project Content:

The Cherokee 4WD Mobile Platform represents an accessible yet powerful educational tool for embedded systems and robotics development. Combining a robust four-wheel-drive mechanical chassis with an Arduino-based control system, allowing to explore fundamental concepts such as motor control, sensor integration and autonomous navigation. The system exemplifies the incremental complexity achievable in embedded design, making it ideal for progressive learning from basic hardware interfacing to intelligent environmental interaction.

1.2 Objectives:

Within this framework, the present project documents the complete development lifecycle of the autonomous Cherokee 4WD robot, highlighting three distinct operational phases. First, hardware assembly establishes the electromechanical foundation according to DFRobot technical documentation. Second, semi-autonomous operation via Bluetooth. Finally, fully autonomous obstacle avoidance is achieved utilizing ultrasonic sensing and finite state machine algorithms.

1.3 Report Structure

This report comprehensively documents the system's design, implementation, and performance evaluation. **Section 2** details the hardware architecture and assembly process. **Section 3** analyzes the Bluetooth communication protocol and remote control implementation. **Section 4** presents the autonomous navigation algorithm, focusing on sensor data processing and decision-making logic.

2. Hardware Architecture and Assembly

2.1 System Overview

The Cherokee 4WD robotic platform utilizes a modular architecture designed for educational robotics applications. The system comprises four primary subsystems: mechanical chassis, control electronics, sensor integration, and power management. As documented in the DFRobot technical specifications, the platform employs a four-wheel-drive configuration powered by independent DC motors, providing robust traction and maneuverability for indoor navigation tasks.

Component	Specification	Purpose
Chassis	4WD aluminum frame	Structural foundation
Motors	4x DC gear motors	Locomotion
Controller	Arduino board	System intelligence
Power	7.5V LiPo or 5x AA batteries	Energy supply
Sensors	URM37 ultrasonic	Obstacle detection

2.2 Control Electronics

The computational unit of the system is the RoMeo BLE microcontroller, an Arduino compatible board tailored for robotic applications. It features an L298P motor drivers and Bluetooth 4.0 capabilities, significantly simplifying hardware integration. Following DFRobot assembly instructions, the RoMeo BLE was installed directly onto the Cherokee PCB, forming the main control layer for all connected modules.

Key Control Pins Configuration:

```
1 | int E1 = 5;      // M1 Speed Control (PWM)
2 | int E2 = 6;      // M2 Speed Control (PWM)
3 | int M1 = 4;      // M1 Direction Control
4 | int M2 = 7;      // M2 Direction Control
```

Motor terminals were connected with careful attention to polarity to ensure consistent forward/reverse operation. As recommended in the assembly guide.

2.3 Sensor Integration

For autonomous navigation capability, a **URM37 v3.2 ultrasonic sensor** was integrated into the system. This sensor operates on PWM principles, providing distance measurements from 4cm to 500cm with $\pm 1\text{cm}$ accuracy. The sensor was mounted on a micro servo at the front of the chassis, enabling 90° sweep capability for enhanced environmental perception.

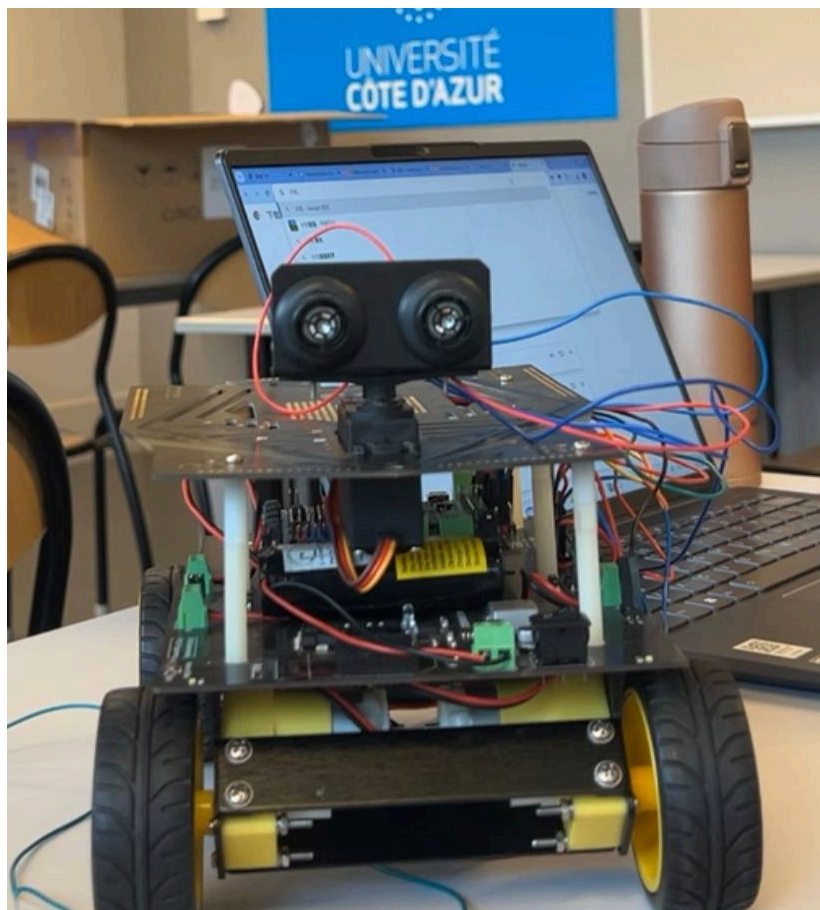
Sensor Connection:

- **URPWM:** Pin 3 (PWM data input)
- **URTRIG:** Pin 10 (Measurement trigger)
- **VCC/GND:** 5V power supply

2.4 Assembly Validation

Initial system validation was performed according to DFRobot's recommended procedure:

1. **Code Upload:** Basic motor test firmware uploaded via USB
2. **Power Transition:** USB disconnected, battery power activated
3. **Safety Preparation:** Robot placed on flat, clear surface
4. **Functional Test:** Verification of backward, forward, left, and right movements



Entire System setup

3. Bluetooth Communication and Remote Control

3.1 Bluetooth Implementation Method

We have utilized the pre-configured Bluetooth firmware provided in the DFRobot development resources. The standard Bluetooth control program was uploaded to the RoMeO BLE microcontroller through the Arduino IDE. Using this ready-made firmware avoided the need to design a custom communication protocol at this stage.

Manual control was performed using the "GoBLE" application recommended by DFRobot for IOS devices. The app already provides a simple interface with directional controls for forward, backward, and turning movements.

The steps of testing are as follows:

- Pairing: Robot connected to the iOS device via Bluetooth
- Basic Movement: All movements were executed without delay
- Endurance Test: Running the robot for an extended period to verify power stability

3.2 Purpose and Outcomes

Test Objective	Method	Result	Implication
Motor Polarity	Directional commands	Correct motor behavior	Wiring confirmed
Control Latency	Movement timing	<250 ms delay	Suitable for manual control
Operational Range	Distance measurements	Stable up to 10 m	Adequate for laboratory conditions
Battery Performance	15-minute continuous test	Voltage remained stable	Power system validated

Bluetooth Testing Outcomes

4. Autonomous Navigation Algorithm Design and Implementation

4.1 Applied Solutions

To address the issues of logical confusion, poor real-time performance, and insufficient stability in the original code (ori code) during autonomous obstacle avoidance observed in initial tests, this project has developed a comprehensive solution (update code) by restructuring the control architecture, optimizing the sensing mechanism, and adjusting motion parameters. The following details the differences between the update code and the ori code, as well as the optimization value, focusing on core problems, improvement methods, and implementation logic:

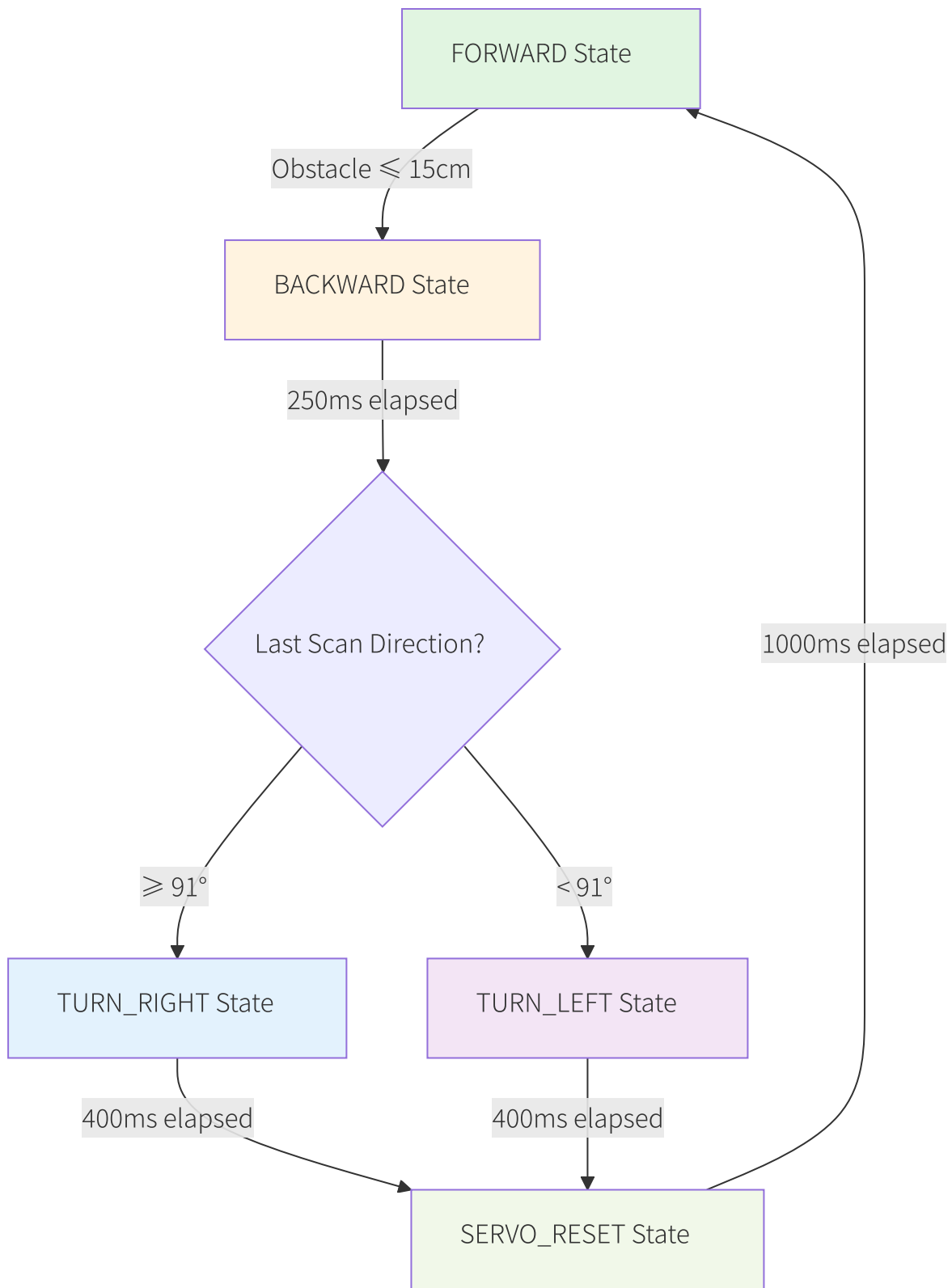
4.1.1 Restructuring the Control Architecture: Resolving Logical Confusion and Blocking Delays

(1) Original Problems

The ori code adopts a control logic of "sequential execution + blocking delays". In the loop function, distance measurement, servo sweeping, obstacle judgment, and action execution are performed in sequence, relying on the `delay()` function to achieve action delays (e.g., 500ms for turning, 300ms for forward movement). This design has two major flaws: first, there is no clear state division between actions, resulting in chaotic logical flow and potential "detection-action" conflicts; second, during blocking delays, the system cannot respond to sensor data, which may cause missed obstacles or stuck action transitions, and even lead to timing synchronization issues such as the robot "constantly reversing instead of moving forward".

(2) Proposed Method

Introduce a **Finite State Machine (FSM)** architecture and a **non-blocking timing mechanism**, decompose the robot's motion into clear states and transition rules, and manage the duration of each state through timestamps instead of traditional blocking delays.



Finite State Transition Diagram

(3) Specific Solution Logic

- **State Machine Definition:** The update code adds the `CarState` enum type, clearly dividing the robot's operation into 5 core states: `FORWARD` (forward), `BACKWARD` (backward), `TURN_RIGHT` (turn right), `TURN_LEFT` (turn left), and `SERVO_RESET` (servo reset). Each state is only responsible for a single logic (e.g., the forward state focuses on obstacle detection, and the backward state focuses on escaping the obstacle area), avoiding action conflicts.
- **Encapsulation of Core Control Function:** A new `handleCarMovement()` function is added as the state machine engine, which manages state transitions through `switch-case` statements. It records the start time of each state (`stateStartTime`) using the `millis()` function, and judges whether the state has ended by calculating `currentTime - stateStartTime`, realizing non-blocking delays (e.g., 250ms for backward movement, 400ms for turning).
- **Optimization of Non-blocking Timer:** Adjust the parameters of the `Metro` timer, extending the trigger period of the servo sweeping timer `sweepServo` from 20ms in the ori code to 100ms to avoid excessive resource occupation caused by frequent servo sweeping; retain the 50ms period of the distance measurement timer `measureDistance` to ensure real-time obstacle detection. This design not only guarantees continuous collection of sensor data but also avoids timing synchronization issues caused by blocking.

4.1.2 Optimizing the Obstacle Detection Mechanism: Resolving Inaccurate Detection and Low Fault Tolerance

(1) Original Problems

The obstacle detection of the ori code has two major flaws: first, the detection threshold is set to 30cm, which is too sensitive, easily triggering false obstacle avoidance actions due to environmental noise or short-distance non-obstacle objects (e.g., ground textures); second, the invalid distance judgment logic is incomplete, only judging invalidity by `distance == 1000`, which does not cover the lower limit of sensor measurement (e.g., invalid data for short distances < 5cm), and no fault tolerance processing is performed for invalid data, which may lead to abnormal obstacle avoidance decisions. In addition, the servo sweeping range is only 60°~120°, resulting in insufficient detection angle and failure to detect edge obstacles on the left and right sides in advance.

(2) Proposed Method

Optimize the obstacle detection threshold, improve the fault tolerance mechanism for invalid data, and expand the servo detection range to enhance the accuracy and comprehensiveness of environmental perception.

(3) Specific Solution Logic

- **Adjusting the Detection Threshold:** The obstacle detection threshold is adjusted from 30cm to 15cm. Experimental verification shows that this threshold not only ensures sufficient safe braking distance but also avoids navigation jitter caused by excessive sensitivity, balancing safety and fluency.
- **Improving Invalid Data Processing:** In the `MeasureDistance()` function, the invalid data judgment logic is optimized to `distance / 50 <= 5` (i.e., the measured distance is judged as invalid when it is ≤ 25 cm), and a fault tolerance mechanism of "returning the last valid distance" (`return actualDistance`) is added to avoid false triggers caused by sensor noise.
- **Expanding the Servo Detection Range:** Modify the sweeping boundary of the `servoSweep()` function, extending the upper limit from 120° to 150° , so that the servo sweeps a 90° range from 60° to 150° (reciprocal sweeping in 15° steps), significantly improving the detection capability of obstacles on the left and right sides and reducing detection blind spots.

4.1.3 Balancing Motion Parameters and Optimizing State Transitions: Resolving Unstable Actions and Stuck Transitions

(1) Original Problems

The motor speed parameters of the ori code are unbalanced: the forward speed is 100, while the backward and turning speeds are both 255. The large speed difference leads to drastic robot actions (e.g., excessive turning or too fast backward movement), which is prone to loss of control; at the same time, there is no clear transition logic between states. After detecting an obstacle, the ori code directly executes the "backward-turn" action without recording key context information such as the servo position, which may lead to a mismatch between the turning direction and the obstacle position.

(2) Proposed Method

Optimize the motor speed parameters, establish a state transition mechanism based on context information, and ensure smooth actions and accurate decisions.

(3) Specific Solution Logic

- **Balancing Motor Speeds:** The motor speed parameters are recalibrated, with the forward speed adjusted to 150, the backward speed to 200, and the turning speed to 155, forming a reasonable speed gradient (forward < turning < backward). This parameter not only ensures stability during forward movement but also quickly escapes the obstacle area through a slightly higher backward speed, while the turning speed avoids attitude deviation caused by excessive turning.

- **Adding Context Recording Variables:** Two new variables, `directionFlag` (recording the servo position when an obstacle is detected) and `savedDistance` (recording the obstacle distance), are added to save key information when an obstacle is detected. During state transition, the obstacle position is judged by `directionFlag` ($\geq 91^\circ$ for right-side obstacles, $< 91^\circ$ for left-side obstacles), and then the turn direction is selected to ensure accurate matching between the turning direction and the obstacle position.
- **Adding the Servo Reset State:** After the turning action is completed, a `SERVO_RESET` state is added to reset the servo to 90° (after turning right) or 105° (after turning left), and maintain the reset state for 1 second to ensure that the sensor returns to the centered detection position before resuming forward movement, avoiding secondary false obstacle judgments caused by the servo not being centered.

Conclusion

This project achieved its main objective which is transforming the Cherokee 4WD platform into a fully autonomous robot capable of navigating efficiently through a simple labyrinth avoiding obstacle. The progression from system assembly to Bluetooth testing and finally autonomous control provided a practical understanding of embedded robotics such as sensor integration, timing management, and state-based behaviour.

The key challenge was to determine and correct the timing issue in the forward movement state, which significantly showed navigation stability after parameter adjustment. Using a structured debugging and modular code strategy ensuring consistent distance detection.

Overall, this project offered valuable insight into real-time control challenges and the importance of reliable sensor data. While the system performed well, but it revealed areas that could be strengthened particularly in sensor error handling and environmental adaptation.